

Asymptotic

Incremental Repository Audit

Evidence integrity, publication correctness,
multi-pass build provenance, and analytic boundaries

Repository [VladimirReshetnikov/Asymptotic](#)

Reviewed snapshot 8e859961d7d37f008b826f3a8cad406460271614

Review date 10 September 2026

Deliverables Article, runnable tests, source fragments, candidate guards

Evidence at a glance

Four scoped tooling findings; one build-cache policy improvement; one unresolved analytic contract question. Twenty-two focused Python tests passed, including a three-pass real `pdflatex` recorder experiment. No Wolfram Language or Mathics package execution was available in this session.

This is an incremental review, not a recitation of the existing four-wave backlog. Candidate code is supplied separately from the reviewed source and is not represented as an integrated or accepted repository patch.

Abstract

This review examines a pinned snapshot of the Asymptotic repository, with particular care not to recycle the extensive findings already recorded in its external code-review collection. Source inspection covered mathematical kernels, Mathics adaptations, native-series import, exact inverse certificates, Fourier coefficients, special-function routes, and validation and publication tooling. The clearest additional problems found are in the supporting evidence and publication pipeline: historical test reconciliation can treat a changed test suite as a correction of an old failure; PDF synchronization changes ordinary POSIX access permissions; multi-pass build provenance retains only the final recorder observation; and independently launched builders can overwrite each other’s successful receipt entries despite atomic JSON replacement.

The article supplies counterexamples, execution boundaries, candidate repairs, and acceptance criteria for these four issues. It also separates historical artifact identity from freshness under the current TeX environment, and analyzes an unresolved complex-phase assumption in the native-series importer without claiming a demonstrated public-API failure. Positive mathematical checks explain why several tempting complaints about the special-function and Fourier code were not promoted to findings. The accompanying 22-test suite executes extracted Python behavior and independent fixtures, not the full repository or either symbolic kernel. Novelty screening used the maintained four-wave indexes and consolidated registers; it was not a fresh cover-to-cover rereading of all 36 original articles.

Contents

1	Executive assessment	3
2	Snapshot, inspection scope, and exclusions	3
2.1	Pinned source and access	3
2.2	Source coverage	4
2.3	What the non-duplication screen establishes	5
3	T01: Reconciliation does not preserve test-contract identity	5
3.1	Source mechanism	5
3.2	Minimal counterexample and consequence	5
3.3	Repair policy	6
3.4	Acceptance criteria and migration	6
4	T02: Atomic PDF synchronization changes POSIX access permissions	7
4.1	Source mechanism and executed observation	7
4.2	Why atomic replacement is not the repair	7
4.3	Candidate implementation and limits	7
4.4	Acceptance criteria	8
5	T03: The build receipt records only the final TeX pass’s inputs	8
5.1	The missing temporal dimension	8
5.2	An executed three-pass witness	8

5.3	Repair and evidence representation	9
5.4	Cost and acceptance	9
6	T04: Atomic JSON replacement does not prevent lost receipt updates	9
6.1	A two-writer execution allowed by the source	9
6.2	Consequence and scope	10
6.3	Two coherent repairs	10
6.4	Acceptance criteria	10
7	P01: Separate historical artifact identity from environment freshness	11
8	Mathematical audit: what was checked and what remains uncertain	11
8.1	Why no additional public kernel bug is claimed	11
8.2	Exact special-function normalization retains necessary sectors	12
8.3	Parameterized incomplete-gamma offsets have the right orientation	12
8.4	A constructive check of the Lerch remainder majorant	12
8.5	The first nonlinear Fourier inverse coefficient checks out	13
8.6	Q01: A real retained phase is not a real full phase	14
9	Implementation plan and development opportunities	14
9.1	A focused order of work	14
9.2	A concrete contract model for supporting artifacts	15
9.3	Performance work justified by these findings	15
9.4	API and user-experience changes	15
10	Evidence, reproduction, and artifact guide	16
10.1	What actually ran	16
10.2	Reproduction commands	16
10.3	Archive contents and their intended use	16
11	Limitations and final recommendation	17

1 Executive assessment

The highest-value new work identified here is to make the repository’s evidence and publication contracts as precise as its mathematical contracts. The implementation has already received extensive review and substantial subsequent changes. Many initially plausible complaints about cutoffs, assumptions, Mathics evaluation, numerical precision, and resource limits are already present in the maintained registers. They are not repeated here as new findings or as a recycled development roadmap. The current four-wave index contains 36 review packages and describes 231 attributed entries before overlap is grouped; those counts are neither a current defect count nor an acceptance measure.[2, 3, 4]

The new findings concern observable, narrower behavior:

ID	Priority	Finding	Evidence level
T01	High for evidence	Reconciliation equates a repeated test ID with the same test contract, despite changed suite identity.	Extracted function; synthetic receipts
T02	Medium	PDF synchronization replaces an existing readable file with an owner-only file on POSIX.	Executed publication sequence
T03	Medium	Three-pass build provenance retains only the final pass’s recorder inputs.	Real three-pass TeX experiment
T04	Medium	Concurrent builder instances can lose successful entries in the shared build ledger.	Extracted writer; deterministic interleaving

Priority is local to the affected contract. T01 is not a mathematical wrong-answer defect or evidence of falsified historical results. T02 does not change PDF contents; it changes who can read the replacement. T03 is not a claim that a supplied paper was built from unauthorized material. T04 requires multiple writer processes: the serial loop in one invocation is not itself incorrect. These qualifications are essential to keep the findings useful.

Two additional items are deliberately outside that defect count. P01 proposes an explicit build-cache freshness policy for installed TeX inputs and tools. The present predicate is defensible as a check of an existing historical artifact, but not as a check that rebuilding in today’s environment would produce the same result. Q01 is an analytic contract question concerning a real retained phase with an unspecified complex tail. Its mathematical countermodel is valid, but reachability through native **Series** and the public package dispatcher was not established.

The supplied candidate guards implement conservative reconciliation identity, POSIX permission preservation, three-pass recorder aggregation, and a cooperative locked merge of individual receipt entries. They are intentionally small. They do not replace the package, change the user’s repository, claim Windows ACL support, or prove that a PDF and its receipt are committed as a single transaction. Each integration boundary is stated below.

2 Snapshot, inspection scope, and exclusions

2.1 Pinned source and access

The reviewed commit is `8e859961d7d37f008b826f3a8cad406460271614`, whose recorded commit timestamp is 10 September 2026 at 04:00:14 UTC. Its message is “Publish paired wave-4 runtime checks and

native preservation evidence.” All repository citations in this article resolve to this immutable source state. Subsequent changes to `main` are outside the review.^[1]

Repository contents were read through the GitHub connector. A complete local checkout was not obtained. Selected short Python functions were manually transcribed into `fixtures/upstream_fragments.py`; that file documents the source functions, the supplied imports, and the limited fixture adapters. In particular, it does not emulate upstream Windows path handling. The PDF publication witness executes the relevant source sequence, not the complete `sync_article` validator.

The Wolfram connector returned service errors, and no functioning local Wolfram or Mathics runtime was available. Consequently, the article never attributes a Python result, TeX experiment, mathematical derivation, or historical repository receipt to a fresh symbolic-kernel run. The preserved repository evidence is informative about its own recorded source and scope, but was not independently regenerated here.

2.2 Source coverage

The inspection was broader than the eventual list of findings. The following map describes the principal areas inspected; it does not imply that every file or every branch in an area was exhausted.

Area	Representative inspected source	Review result
Core inversion	Selected ranges of <code>AsymptoticAnalysis.wl</code> ; <code>IncrementalInverse.wl</code>	Formula and state-invariant inspection; no additional confirmed kernel defect
Mathics adaptation	Compatibility, assumptions, Taylor, simplification, calls, time budget, calculus, core functions, and certificate adapters	Source/evaluation-contract analysis; no runtime compatibility claim
Exact certificates	<code>InverseCertificates.wl</code>	Interval and retry-path inspection; no new counterexample established
Fourier inversion	<code>FourierCoefficients.wl</code>	Coefficient, convolution, residual, and coordinate analysis
Special functions	Identities, parameterized functions, Dirichlet functions, native series, and real-domain modules	Exact-identity and remainder checks; Q01 remains unresolved at the public boundary
Test evidence	Acceptance checker, historical summarizer, summary tests, native-definition capture/comparison, Mathics workflow	T01; other candidates excluded or not substantiated
Documentation and publication	Guide builder, local-link checker, ProveIt PDF builder and synchronizer	T02–T04 and P01
Prior-review control	Four-wave index, wave-4 index, consolidated status, wave-3 and wave-4 intake ledgers	Novelty screen and exclusion of recorded backlog items

2.3 What the non-duplication screen establishes

The maintained registers were used as the primary exclusion set. The reported T01–T04 mechanisms were not identified there as existing findings. T01 is about identity of the assertion being rerun, not about files changing during a run. T02 is about publication permissions, not output-path aliasing. T03 is about separate dependency observations across TeX passes, not the Mathics loader’s input closure. T04 is a multi-writer lost-update problem, not interrupted single-process receipt writing. These distinctions are narrower than generic advice to improve provenance or make writes atomic.[4, 5, 6]

There is still a limit: all 36 original articles and every attached code file were not reread in full. An incomplete GitHub code-search response was not used to prove absence. Thus “additional” in this article means additional relative to the inspected indexes and consolidated finding registers, not a certified absence of any semantically similar sentence anywhere in the archive. The archive’s own index also distinguishes its catalogs from exhaustive rereading and validation.[3]

To respect the user’s exclusion, no new finding is assigned to the broad recorded C-, P-, D-, V-, X-, W3-, or W4-backlog items merely because the present source still exposes their symptoms. Previously proposed mathematical feature families are likewise not restated as new development opportunities.

3 T01: Reconciliation does not preserve test-contract identity

3.1 Source mechanism

The historical summarizer deliberately preserves original receipts and their original failure counts. That is a sound design choice. The problem lies in what it calls a correction. In `validation/summarize_mathics_tests.py`, `read_receipt` retains `TestSuiteSnapshotSHA256`, while `package_hashes` extracts only package files. The `reconcile` function compares the package-file map and checks that all original failing IDs appear among successful correction IDs. It never compares the retained suite identity.[7]

The decisive logic is equivalent to:

```
if full["PackageSourcesSHA256"] != correction["PackageSourcesSHA256"]:
    raise ValueError(...)
unresolved = set(full["FailedTestIDs"]) - set(correction["SuccessfulTestIDs"])
if correction["FailedTestIDs"] or unresolved:
    raise ValueError(...)
# The pair is now called a correction.
```

The existing test suite explicitly verifies that changing the test-suite path fingerprint does not change the *package* fingerprint. That separation is correct; conflating it with test-contract identity is not. The current tests also protect original failure counts and reject changed package sources, but do not require assertion identity for a correction.[8]

3.2 Minimal counterexample and consequence

Let an unchanged package return an incorrect result in a test with ID `same-id`. Receipt A records failure of an assertion that the result should be zero. The suite is then edited so that the same ID checks a weaker condition, or simply expects the observed nonzero result. Receipt B records success. Both receipts can be immutable, honestly generated, and correctly identify exactly the same package bytes. They nevertheless concern different propositions.

The supplied fixture gives A and B different `TestSuiteSnapshotSHA256` values and otherwise supplies the fields used by `reconcile`. The transcribed function returns `CorrectedTestIDs=["same-id"]`. The witness is not an allegation that the repository’s historical normalization reruns weakened any assertion. It shows that the summarizer cannot establish that they did not.

Formally, if P denotes package source, T a test contract, and $R(P, T)$ its outcome, then

$$P_A = P_B, \quad \text{ID}(T_A) = \text{ID}(T_B), \quad R(P_A, T_A) = \text{fail}, \quad R(P_B, T_B) = \text{pass}$$

does not imply $R(P_A, T_A)$ has been independently corrected. Test names are labels, not definitions of the predicates they name. The difference can arise without an adversary: expected-output normalization, harness maintenance, a changed simplification oracle, or a revised tolerance can change what was actually checked.

The newer complete-acceptance verifier binds evidence to a reference suite and runner. That stronger route is not the target of this finding. Its existence should not be read backward into the older summarizer’s reconciliation semantics.^[9]

3.3 Repair policy

For automatic reconciliation, require equality of package identity, test-suite identity, and runner identity. Preserve the raw receipts unchanged. When the suite or runner differs, classify the later result as supplemental successful evidence rather than as automatic resolution of an earlier failure. The candidate `require_same_test_contract` performs this conservative comparison. Its runner field is a *proposed normalized field*; upstream `read_receipt` must populate it from validated runner provenance before the guard can be integrated.

A whole-suite identity check is intentionally conservative. Adding an unrelated test can change the suite without changing the failing case. A future refinement can store per-case contract identities, including the assertion, expected-value computation, harness semantics, and relevant options. Such an identity should not be merely a digest of the visible test body if helper definitions can change its meaning. A reviewed equivalence record can authorize a changed assertion, but that record must remain explicit and separate from the fact that both runs used the same package.

3.4 Acceptance criteria and migration

A changed-suite same-ID pair must not receive an automatic correction status. An unchanged package, suite, and runner must remain eligible for the existing successful-rerun checks. Missing identity in legacy receipts must produce an unknown or migration-required status, not an invented identity. Existing original counts and failure IDs must remain visible in every outcome.

Historical records should not be rewritten to erase earlier summaries. Publish a new interpretation that distinguishes “same contract rerun,” “changed contract with reviewed equivalence,” and “supplemental result.” This corrects the meaning of the evidence without accusing earlier producers of misconduct or discarding useful experiments.

4 T02: Atomic PDF synchronization changes POSIX access permissions

4.1 Source mechanism and executed observation

In `validation/sync_proveit_pdfs.py`, `sync_article` creates a file with `tempfile.mkstemp` in the destination directory, writes validated PDF bytes, flushes and synchronizes them, rechecks its evidence, and then calls `os.replace`. No permission-copy or permission-policy step appears before the replacement.^[10]

The operations tested are:

```
descriptor, name = tempfile.mkstemp(
    prefix=".proveit-pdf-", suffix=".tmp", dir=target.parent)
with os.fdopen(descriptor, "wb") as stream:
    stream.write(data)
    stream.flush()
    os.fsync(stream.fileno())
os.replace(name, target)
```

Python documents `mkstemp` files as accessible only to their creating user. On the POSIX test platform, an existing destination with mode `0644` became `0600` after this sequence. The replacement bytes were correct; the access metadata was not preserved.^[12]

This is precisely why byte-level checks alone do not establish publication correctness. A shared reader, web-serving identity, or collaborating account that could read the previous file may no longer be able to read its replacement. The damage is conditional on the deployment’s access model; it is not a demonstrated failure of a particular hosted service. It also need not appear in a usual Git diff of an ordinary non-executable file’s contents.

4.2 Why atomic replacement is not the repair

The source already uses atomic replacement for the file’s directory entry. That is valuable, but it publishes the temporary file’s metadata together with its bytes; it does not promise to inherit all metadata from the old file.^[13] The missing contract is therefore not “use an atomic write.” It is “what access policy should the newly published artifact have?”

For an existing ordinary POSIX file, the conservative default is preservation of its read/write/execute permission bits, with an explicit rejection or policy for unusual special bits. For a new file, the application must select a policy: private by default, or an explicitly requested public mode. Unconditionally calling `chmod(..., 0644)` would avoid this particular loss but would also silently broaden access to intentionally private files.

4.3 Candidate implementation and limits

The supplied `atomic_publish_posix_mode` obtains the existing target’s mode, rejects symlinks and nonregular existing destinations, writes to a temporary file, applies the chosen mode through the open descriptor, synchronizes the file, replaces the destination, and synchronizes its parent directory. Tests cover mode preservation, explicit new-file policy, private defaults, and symlink rejection.

This is a POSIX-mode candidate, not a universal metadata-preservation function. Ownership, ACLs,

extended attributes, security labels, hard-link semantics, and Windows access-control inheritance require a separate contract. Nor does the candidate eliminate a race with an uncooperative writer between its inspection and replacement. The existing application validation and the publication synchronization policy must surround it. The limitation is documented in the function itself rather than hidden behind the word “atomic.”

4.4 Acceptance criteria

At minimum, replacing a 0644 PDF must preserve 0644, replacing an intentionally private PDF must not make it public, and new-file access must be explicitly defined. Verify the real synchronizer on both a successful update and an `already_current` path: the latter should not rewrite the file or its metadata. A Windows test must exercise its actual ACL policy, not infer it from the POSIX tests in this archive.

5 T03: The build receipt records only the final TeX pass’s inputs

5.1 The missing temporal dimension

The PDF builder executes three `pdflatex` passes. It preserves separate console and log files for each pass, but calls `recorder_inputs` only after the loop, reading the final `.fls` file. The function’s input classification therefore describes that surviving observation, not the union of all inputs read during the build.^[11]

Write I_j for the actual input set of pass j . The dependency observation for a three-pass computation is

$$I_{\text{build}} = I_1 \cup I_2 \cup I_3,$$

not I_3 . Equality is a special case, not an invariant of multi-pass TeX. The full logs remain valuable historical evidence, but retaining them is not equivalent to producing a complete structured recorder inventory. This finding is about the meaning and completeness of that inventory, not about erasing the raw logs.

5.2 An executed three-pass witness

The included experiment creates a fresh TeX mirror and an external TeX input. A minimal document reads the external input only before its auxiliary file exists:

```
\documentclass{article}
\IfFileExists{probe.aux}{}{\input{/temporary/path/first-pass-only.tex}}
\begin{document}Recorder experiment.\end{document}
```

The test runs the same three-pass shape as the builder and observes each recorder immediately. The external file is present in the first observation and absent in the third. The transcribed `recorder_inputs` function correctly classifies the file when given the first recorder; the loss occurs because `build_one` does not collect that observation. This is a real TeX experiment, not a manufactured parser output. It is still not a complete run of the repository’s vendor builder.

An analogous pattern can arise in ordinary build logic that generates or consults auxiliary state. The experiment need not demonstrate that a particular vendored article presently uses the pattern: it disproves the assumption that the third pass subsumes all earlier inputs.

5.3 Repair and evidence representation

Capture the recorder after *each* successful pass, while that pass’s observation still exists. Preserve a pass-indexed record and compute the union used for build-level provenance and closure checks. If the same external path is observed with different contents in different passes, do not choose the last identity silently. Record or reject the inconsistent environment according to the build contract.

The supplied `merge_pass_recorders` consumes three observations, retains all three, and forms a deterministic union. It rejects a repeated runtime path with inconsistent recorded content. It is an aggregator, not a time machine: calling it three times on the last `.fls` file cannot recover the lost observations. Integration must move the call to `recorder_inputs` inside the pass loop. The declared vendored closure should be checked against the aggregated input set rather than only the final one.

Not every generated intermediate belongs in a stable source manifest. The existing distinction between vendored source, installed/runtime input, and mirror-generated auxiliary state is useful. The change proposed here preserves that distinction while adding the missing pass coordinate. It does not demand that generated `.aux` files be misclassified as immutable source inputs.

5.4 Cost and acceptance

There is extra I/O because installed files may be inspected after each pass. The safe optimization target is duplicate work *within an observation*, not deleting earlier observations. Reusing a final-pass file identity to stand for an earlier read would reintroduce the same evidentiary gap. First implement the complete observation semantics; then measure actual overhead before adding a carefully specified cache.

The real first-pass-only fixture must be retained by the final build record. A dependency used only in pass two must also be retained. The same path with changed contents across passes must be distinguished from a stable dependency. A normal document with identical inputs across all three passes should still produce a compact deterministic union and three explicit observations.

6 T04: Atomic JSON replacement does not prevent lost receipt updates

6.1 A two-writer execution allowed by the source

The builder reads `builds.json` into a dictionary, passes that dictionary through its serial article loop, updates it after each successful build, and writes the whole dictionary with an atomic temporary-file replacement. No cross-process lock or fresh-read merge surrounds the read–modify–write sequence.^[11]

Consider two independently launched invocations selecting different articles. Each reads an initially empty ledger L_0 . Invocation A finishes article A and writes

$$L_A = L_0 \cup \{A \mapsto r_A\}.$$

Invocation B, still using its own old dictionary, finishes article B and writes

$$L_B = L_0 \cup \{B \mapsto r_B\}.$$

The final ledger lacks A even though A’s PDF has already been published. Both JSON replacements can be atomic and every individual JSON file can be valid. Atomicity of a write does not serialize an earlier read and modification.

The test uses two copies of the same initial state and the transcribed `write_json`. It deterministically executes the two publications in this order and observes only B in the final ledger. This is a minimal interleaving model of the actual source algorithm, not a stress test of two complete TeX builder processes. It avoids pretending that a nondeterministic test must fail on every run to establish a lost-update schedule.

6.2 Consequence and scope

This is conditional on multiple builder processes. The module describes its build loop as serial, and one process following that loop does not trigger the counterexample. The tool does not, however, enforce exclusive writer ownership of the vendor tree. A second terminal, an overlapping automation job, or a separate article build can create the schedule.

The lost record is recoverable from other artifacts or by rebuilding, but the shared ledger ceases to be a reliable catalog of completed work. The planner can classify an otherwise successful artifact as unverified, and the synchronizer can refuse publication because the completed receipt is missing. These consequences follow from the planner’s and synchronizer’s receipt requirements; no actual production loss is asserted.[11, 10]

6.3 Two coherent repairs

The simplest supported policy is a single-writer lock around the complete build invocation. It should fail clearly or wait according to an explicit policy when the vendor tree already has a writer. This preserves the current in-memory ledger algorithm at the cost of serializing all invocations, not just the article loop within each invocation.

A more flexible design builds expensive mirrors without holding the ledger lock, then acquires a publication lock, rereads the current ledger, validates the target article’s expected prior state, and commits one article update. Different-article updates merge; conflicting same-article updates are rejected or resolved explicitly. The supplied `commit_receipt` demonstrates the fresh-read, lock, and per-article compare-and-swap portions on POSIX. It uses `flock`; all writers must cooperate.[14]

The function is not a full publication transaction. A correct integration must place the relevant PDF replacement and receipt update under the same publication policy, or publish immutable PDF/receipt objects and switch an article pointer atomically. Merely replacing the final JSON-writing call after the PDF has already changed would leave a same-article conflict window. Similarly, failure records must merge into a freshly read ledger; an exception path that writes an old entire dictionary can undo a successful concurrent commit just as readily as a success path can.

6.4 Acceptance criteria

Two independent article updates must survive in the shared ledger, regardless of which build finishes first. A same-article stale expected value must not replace a newer successful record. A failing process must not remove another process’s success. Cancellation and stale-lock handling must not encourage unlinking an active lock file and creating a different lock identity. Test the actual chosen policy on the operating systems the repository supports; the included prototype establishes only the stated POSIX behavior.

7 P01: Separate historical artifact identity from environment freshness

This is an API and build-policy improvement, not an additional unconditional bug. In `receipt_matches`, the builder checks the recorded source-input bytes, three successful pass-log identities, and PDF bytes. It requires a nonempty `tool_versions` record, but does not compare recorded tools with the current tools. Nor does it revalidate the installed/runtime inputs stored by the recorder. When the predicate succeeds and no forced rebuild is requested, `plan_article` returns a skip decision.^[11]

The included schema-level fixture records a runtime style input, changes its contents, and observes that `receipt_matches` remains true. This result is consistent with the predicate’s actual checks. It does not show that the existing PDF changed or that the historical build was invalid.

The ambiguity is the word “current.” There are at least three distinct questions:

H : Does the artifact still match its recorded completed build?

E : Are the recorded build dependencies unchanged in this environment?

B : Does an independently repeated build reproduce the artifact?

H does not imply *E*, and *E* alone need not imply byte-for-byte *B* in the presence of uncontrolled build metadata. The existing predicate addresses a substantial part of *H*; it should not be silently repurposed as *E*.

A useful interface would return separate statuses, for example `ArtifactIdentity`, `EnvironmentFreshness`, and `RebuildReproducibility`. An explicit freshness policy can preserve an archival build without reinstalling its historical environment, or request a rebuild after installed style files and tools change. Missing historical paths would then yield “not comparable” rather than a false assertion that the artifact is corrupt.

T03 should precede an environment-freshness feature: a policy cannot validate all build-time dependencies from an inventory that omitted earlier passes. An environment key also needs declared lookup-path and engine policy, because changing a search path can select a different dependency even when an old recorded file still exists unchanged. This is a concrete, bounded development opportunity in the current PDF workflow, not a proposal to build a general reproducible-build platform before fixing the observed issues.

8 Mathematical audit: what was checked and what remains uncertain

8.1 Why no additional public kernel bug is claimed

The mathematical implementation was inspected, but source inspection alone cannot determine every interaction with Wolfram evaluation, native series normalization, or Mathics rewrites. This article does not manufacture a new kernel defect to make the findings look more balanced. Several algebraic checks support the current formulas; one importer question needs a precise runtime witness. The checks below explain the distinction.

8.2 Exact special-function normalization retains necessary sectors

The exact-normalization module treats identities separately from finite asymptotic expansions. Its half-integer modified Bessel formulas retain both exponential contributions where necessary. For example,

$$I_{1/2}(z) = \sqrt{\frac{2}{\pi z}} \sinh z = \frac{e^z - e^{-z}}{\sqrt{2\pi z}}.$$

A terminating expression multiplying e^z would not by itself establish an exact identity for this function. The module's finite recurrence keeps the second exponential rather than deriving an identity from a truncated dominant expansion. Its negative-order correction also agrees with the alternating half-integer connection term. This source path did not yield a new missing-sector finding.[15]

Likewise, the integer-parameter identity used for $M(a, a + 1, z)$ has the necessary lower-endpoint contribution. For integer $a > 0$, repeated integration by parts in

$$M(a, a + 1, z) = a \int_0^1 e^{zt} t^{a-1} dt$$

gives

$$M(a, a + 1, z) = \frac{(-1)^{a-1} a!}{z^a} \left(e^z \sum_{j=0}^{a-1} \frac{(-z)^j}{j!} - 1 \right).$$

At $a = 1$ this becomes $(e^z - 1)/z$; at $a = 2$ it becomes $2(e^z(z - 1) + 1)/z^2$. The signs and normalization in the inspected formula are therefore not evidence of an error. This is a derivation check, not an executed special-function package test.

8.3 Parameterized incomplete-gamma offsets have the right orientation

The parameterized Frobenius route writes a small-argument contribution as a power times an ordinary analytic function, plus an offset and multiplier. For $a > 0$,

$$\gamma(a, z) = \frac{z^a}{a} {}_1F_1(a; a + 1; -z), \quad \Gamma(a, z) = \Gamma(a) - \gamma(a, z).$$

For a two-endpoint integral, swapping which endpoint varies also swaps the sign of its contribution. Inspection of the returned offset/multiplier choices in `specialParameterizedFrobenius` is consistent with these identities. The regularized cases add the expected division by $\Gamma(a)$. No new normalization or endpoint-sign bug was established in these paths.[16]

This check does not establish that every requested cutoff is attained by all compositions or every symbolic parameter domain is admitted. Those are separate claims, and broad existing precision/admission issues were screened out rather than repeated.

8.4 A constructive check of the Lerch remainder majorant

The Dirichlet module uses moment sums for the large- A Lerch expansion with fixed $|z| < 1$ and real s . The mechanism can be checked directly without a symbolic kernel. Starting from

$$\Phi(z, s, A) = \sum_{m \geq 0} \frac{z^m}{(A + m)^s}, \quad A \geq 1,$$

expand $(1+t)^{-s}$ to degree $n-1$ at zero. For $n \geq 1$, Taylor's integral remainder and

$$\frac{d^n}{dt^n}(1+t)^{-s} = (-1)^n (s)_n (1+t)^{-s-n}$$

give a bound involving the maximum of $(1+t)^{-s-n}$ on $0 \leq t \leq m/A$. Set $d = \max(0, \lceil -s-n \rceil)$. On that interval,

$$(1+t)^{-s-n} \leq (1+m)^d.$$

Consequently,

$$|R_n(A)| \leq \frac{|(s)_n|}{n!} A^{-s-n} \sum_{m \geq 0} |z|^m m^n (1+m)^d.$$

Expanding $(1+m)^d$ turns this into

$$|R_n(A)| \leq \frac{|(s)_n|}{n!} A^{-s-n} \sum_{j=0}^d \binom{d}{j} \mu_{n+j}(|z|),$$

where $\mu_k(q) = \sum_{m \geq 0} q^m m^k$. For positive k this is the negative-index polylogarithm, while the zero moment is $1/(1-q)$. The finite moment bound is the structure implemented in the inspected module.[\[17\]](#)

This derivation explains why a factor $(1+m)^d$ is not gratuitous pessimism: it handles a growing Taylor derivative for sufficiently negative s . When a nonpositive integer parameter makes a higher derivative vanish, the finite polynomial termination is consistent with the vanishing Pochhammer factor. The bound concerns fixed z and s ; it does not assert uniform control as $|z| \rightarrow 1$. No broader feature request is inferred from this check.

8.5 The first nonlinear Fourier inverse coefficient checks out

For a simple representative of the Fourier coefficient ring, let

$$f(u) = u(1 + au^d \sin(\omega \log u)), \quad d > 0.$$

Write $S = \sin(\omega \log z)$ and $C = \cos(\omega \log z)$. An inverse ansatz

$$u = z \left(1 + Az^d + Bz^{2d} + O(z^{3d}) \right)$$

yields, by substitution,

$$A = -aS, \quad B = a^2((1+d)S^2 + \omega SC).$$

Indeed, the coefficient at relative order z^{2d} in $f(u)/z - 1$ is $B + a(1+d)AS + a\omega AC$, which vanishes for these choices.

For the single perturbation, the inspected Euler–Lagrange rule at multi-index two gives

$$\frac{1}{2} \left(\frac{d}{dL} + 2 + 2d \right) (a^2 \sin^2(\omega L)) = a^2((1+d) \sin^2(\omega L) + \omega \sin(\omega L) \cos(\omega L)),$$

in agreement with direct inversion. The logarithmic derivative term is essential; its presence in the source is correct.[\[18\]](#)

This is a check of a nontrivial coefficient identity, not proof of the entire Fourier implementation. It also illustrates why isolated zeros of a sine factor should not be treated as an asymptotic error scale: the relevant coefficient ring has bounded modes, not eventually nonzero individual modes. The existing Fourier source already maintains that distinction.

8.6 Q01: A real retained phase is not a real full phase

Status: an unresolved internal contract question, not a demonstrated public wrong-answer bug. In `specialNativeTree`, the sine/cosine branch accepts the approximation/error pair $\{h(a), r\}$ immediately when the retained phase a is proved real. That early branch does not require `specialNativeSmallQ[r, ...]`. The complex/hyperbolic path below it does require a vanishing absolute error.[19]

A real retained approximation alone is insufficient for a global Lipschitz bound on a phase with an unspecified complex error. Whole-expression realness does not automatically provide the missing fact either: the cosine of a purely imaginary argument is real. Native `SeriesData` describes series coefficients and order information; a separate property is needed to infer that the omitted phase remains real.[20]

Proposition 1. *Knowing $b(u) = a(u) + O(r(u))$ and $a(u) \in \mathbb{R}$ does not in general imply $\cos b(u) = \cos a(u) + O(r(u))$ when b may be complex and r may grow.*

Proof. Take $a(u) = 0$, $b(u) = i/u$, and $r(u) = u^{-5/2}$ as $u \downarrow 0$. Then $|b(u) - a(u)|/r(u) = u^{3/2} \rightarrow 0$, so the assumed error relation holds. However,

$$\cos b(u) = \cosh(1/u), \quad \frac{\cosh(1/u) - 1}{u^{-5/2}} \rightarrow \infty.$$

Exponential growth dominates the polynomial factor. Both the retained phase and the full outer cosine are real, but the claimed error transfer fails. $\square$

This countermodel concerns the abstract information admitted by the helper, not an observed native output. Several evaluation stages can prevent such an input from reaching that branch: native evaluation may rewrite the imaginary phase to a hyperbolic function, produce a different structured result, or refuse the expansion. The current public dispatcher may also reject the source. Without a functioning Wolfram runtime, no assertion is made that any specified public call produces the bad pair.

A conservative candidate rule would allow the complex-phase transfer only when the phase error tends to zero. For small complex δ and real a ,

$$h(a + \delta) - h(a) = \delta \int_0^1 h'(a + t\delta) dt = O(|\delta|)$$

for $h = \sin, \cos$, because the imaginary parts on the segment remain bounded. A nonvanishing-error fast path could remain available with an explicit proof that the *full phase*, not just its retained approximation, is real. This proposal is intentionally not shipped as an automatic source patch.

The required follow-through is narrow: characterize the helper with actual native objects, then seek a public expression whose returned native tree has this shape, and verify whether the outer exact-realness guard permits it. Until then, Q01 must remain outside the confirmed-defect and package-regression counts. The included numerical illustration only supports the elementary countermodel and is not a surrogate for that missing runtime work.

9 Implementation plan and development opportunities

9.1 A focused order of work

Address T01 first because it changes the interpretation of reported evidence, not merely the speed of producing it. The change should be additive: preserve old receipts and introduce explicit reconciliation

status and identity fields. Then address T02, whose POSIX symptom has a small, directly testable repair. T03 should precede P01 so a freshness policy has complete observations to work with. T04 requires a deliberate choice between enforced single-writer operation and concurrent builds with serialized publication.

Q01 belongs to a separate mathematical characterization task. It should neither block the small tooling repairs nor be presented as a resolved soundness bug because a Python countermodel passed. This separation prevents a mixed patch from combining a clear engineering fix with an unvalidated change to symbolic admission behavior.

9.2 A concrete contract model for supporting artifacts

The additional findings suggest separating four facts that are currently easy to blur in prose:

Fact	Specific question
Assertion identity	Did two observations test the same proposition with the same harness semantics?
Build observation	Which source and runtime inputs were actually observed in each pass?
Artifact identity	Do the current bytes still match the completed build record?
Publication state	Are the intended bytes, reader access, and catalog entry published consistently?

This is not a proposal to replace every result object or introduce a repository- wide schema migration at once. Apply the distinctions first to the exact summarizer and PDF functions identified here. Small explicit statuses are more useful than a single broad `Verified` Boolean when different facts can fail independently.

9.3 Performance work justified by these findings

The proposed fixes create two legitimate optimization questions. First, per-pass recorder capture can repeat installed-file inspection. Measure the additional I/O on actual articles while keeping all temporal observations. Second, a single-writer build lock can block independent expensive TeX work. A publication- only lock permits parallel mirrors, but requires a fresh ledger merge and a same-article conflict policy. Optimize those particular operations rather than repeating the already-recorded generic requests to cache symbolic work or speed up sparse coefficient routines.

For ledger scalability, immutable per-attempt receipts plus a compact current article index would reduce whole-ledger rewrites and improve recovery. That is a direct development option for T04, not a prerequisite for its simplest fix. The index remains derived state; rebuilding it from immutable receipts should not require rerunning TeX. The publication policy must still define which successful attempt is current and how a failed or superseded attempt is shown.

9.4 API and user-experience changes

Historical summaries should explain why a result was placed in a supplemental category rather than called a correction. PDF synchronization should report the chosen access policy, not only content identity. Build planning should distinguish “matches a completed historical receipt” from “fresh under the current tool and installed-input policy.” Concurrent-writer rejection should identify the vendor tree and the operation that needs exclusive access, without silently resetting shared state.

These are localized changes to observable contracts. They are not a new request to implement previously proposed asymptotic families, redesign the public Wolfram API, or replace the existing release roadmap.

10 Evidence, reproduction, and artifact guide

10.1 What actually ran

All 22 test methods in `tests/test_review.py` passed in the review container. The populations are separated below; their sum is a count of these focused methods, not a package-acceptance total.

Test population	Methods
Reconciliation source fragment and candidate identity guard	6
POSIX publication sequence and candidate permission policy	5
Recorder aggregation and real three-pass TeX experiment	5
Runtime freshness schema fixture	1
Ledger lost-update schedule and candidate merge behavior	4
Independent analytic countermodel illustration	1
Total	22

Some methods intentionally assert that the old behavior occurs. A passing test named `test_upstream_sequence_replaces_0644_with_0600` means the defect was reproduced, not that the upstream behavior satisfies the desired contract. Other methods test the proposed guards. The raw output preserves the full test names so the distinction is inspectable.

10.2 Reproduction commands

From the extracted archive root, on Python 3.11 or later:

```
python -m unittest discover -s tests -v
```

The recorder experiment requires `pdflatex`; it is explicitly skipped when that executable is absent. POSIX permission and locking tests are explicitly skipped on non-POSIX platforms. A run with skips is not equivalent to the all-22 execution recorded here. No Wolfram license, Mathics installation, network request, or repository write is required for these focused tests.

To rebuild the article from its directory:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-review.tex
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-review.tex
```

10.3 Archive contents and their intended use

`article/asymptotic-review.tex` and its PDF are the report. `fixtures/upstream_fragments.py` contains the transcribed source behavior and narrowly identified fixture adapters. `code/review_guards.py` contains candidate building blocks, not an integrated patch. `tests/test_review.py` supplies

the executed regression and characterization methods. `evidence/test-output.txt`, `evidence/scope.json`, and `evidence/novelty.csv` record results and limits. The root README describes integration order and platform requirements. No checksum file is included.

The fixture license notice preserves the upstream source notice. The archive does not contain the entire repository, compiled symbolic-kernel results, unexecuted tests mislabeled as passing, or credentials for an unavailable service.

11 Limitations and final recommendation

This review found four additional, source-backed problems in evidence and publication tooling, supported by focused execution. It did not establish a new public mathematical wrong-answer defect, a complete Mathics compatibility assessment, or a full-package native acceptance result. Windows access and locking behavior was not validated. The exact integrated repository programs were not run end to end; extracted Python functions and controlled fixtures were.

The novelty comparison is grounded in maintained finding registers and indexes, not exhaustive rereading of all supplied review prose. The candidate guards must be integrated with the real validation, path, publication, and receipt schemas before their focused results can support repository acceptance.

The recommended next change set is consequently small and concrete: preserve test-contract identity when reconciling failures; preserve or explicitly select publication access policy; retain dependency observations for every TeX pass; and serialize shared-ledger publication rather than relying on atomic writes alone. Keep historical artifact validity separate from environment freshness, and keep Q01 as an explicitly unresolved analytic question until its actual native and public-API reachability is demonstrated.

References

- [1] Repository commit [8e859961d7d37f008b826f3a8cad406460271614](#). [Commit metadata and source state](#). Accessed 10 September 2026.
- [2] [Four-wave code-review index](#).
- [3] [Wave-4 package descriptions, evidence boundaries, and catalog notes](#).
- [4] [Consolidated implementation and finding register](#).
- [5] [Wave-3 intake ledger](#).
- [6] [Wave-4 intake ledger](#).
- [7] [Historical receipt summarizer](#): `package_hashes`, `read_receipt`, `reconcile`, and `build_summary`.
- [8] [Historical-summary evidence tests](#).
- [9] [Exact-reference portable acceptance verifier](#): `reference_from_blobs`, `validate_shard`, and `verify_acceptance`.
- [10] [Vendored PDF synchronization](#): `sync_article`, `synchronize`, and publication checks.
- [11] [Vendored PDF builder](#): `write_json`, `receipt_matches`, `plan_article`, `recorder_inputs`, `build_one`, and `main`.
- [12] Python 3.11 documentation, [`tempfile.mkstemp`](#). Secure temporary-file access semantics. Accessed 10 September 2026.
- [13] Python 3.11 documentation, [`os.replace`](#), [`os.fchmod`](#), and [`os.fsync`](#). Accessed 10 September 2026.
- [14] Python 3.11 documentation, [`fcntl.flock`](#). Accessed 10 September 2026.
- [15] [Exact special-function identity normalization](#).
- [16] [Parameterized Frobenius and analytic composition routes](#).
- [17] [Dirichlet and Lerch special-function expansions](#).
- [18] [Finite Fourier-polynomial inverse coefficient and residual implementation](#).
- [19] [Structured native-series importer](#): especially `specialNativeTree` and `specialFunctionForwardExpansion`.
- [20] Wolfram Language documentation, [`SeriesData`](#). Accessed 10 September 2026.